

+++ date = '2026-02-20T20:40:51-08:00' draft = true title = 'Practica4' +++

Practica 4: Paradigma Lógico con Prolog

Universidad Autónoma de Baja California

Facultad de Ingeniería, Arquitectura y Diseño

Materia: 40032 – Paradigmas de la Programación

Docente: M.I. José Carlos Gallegos Mariscal

Grupo: 941

Nombre: Leonel Cajeme Garcia

Matricula: 379154

Grupo: Ing. Software 941

Fecha: 22 de mayo de 2026

Repositorio en GitHub: github.com/leonelcajeme10-eng/Portafolio-Paradigmas

Página estática (GitHub Pages): leonelcajeme10-eng.github.io/Portafolio-Paradigmas/

Introducción

El paradigma lógico es un estilo de programación basado en la **lógica matemática formal**. En lugar de describir *cómo* resolver un problema paso a paso (como en programación imperativa), el programador define *qué* es verdad mediante hechos y reglas, y el motor de inferencia del lenguaje deduce las respuestas.

Prolog (PROgramming in LOGic) es el lenguaje más representativo de este paradigma. Fue desarrollado en 1972 por Alain Colmerauer y Robert Kowalski, y se basa en la **cláusula de Horn**, un subconjunto de la lógica de predicados de primer orden.

Conceptos Fundamentales

Concepto	Descripción	Ejemplo
Hecho	Afirmación incondicional verdadera	<code>persona(ana, 25).</code>
Regla	Afirmación condicional (si...entonces)	<code>es_joven(X) :- persona(X, E), E < 30.</code>
Consulta	Pregunta al motor de inferencia	<code>?- es_joven(ana).</code>
Unificación	Mecanismo de igualación de términos	<code>X = 5</code>
Backtracking	Retroceso automático en la búsqueda	Interno al motor

Primera Sesión: Instalación del Entorno e Introducción a Prolog

1.1 Instalación de SWI-Prolog

SWI-Prolog es la implementación más popular y completa de Prolog. Para instalarlo:

En Linux (Ubuntu/Debian):

```
sudo apt update
sudo apt install swi-prolog
```

En macOS:

```
brew install swi-prolog
```

En Windows:

Descargar el instalador desde <https://www.swi-prolog.org/download>

Para verificar la instalación:

```
swipl --version
```

Para cargar un archivo:

```
swipl archivo.pl
# 0 dentro del intérprete:
?- [archivo].
```

1.2 Hechos y Reglas Básicas

El programa de la sesión 1 ([sesion1_introduccion.pl](#)) establece una base de conocimiento sobre personas y sus características.

Hechos definidos

```
persona(ana, 25).
persona(juan, 30).
es_estudiante(ana).
gusta(ana, logica).
```

Reglas definidas

```
% Una persona es joven si tiene menos de 30 años
es_joven(X) :-
    persona(X, Edad),
    Edad < 30.

% Dos personas son compañeras si ambas son estudiantes y son distintas
```

```
son_companeros(X, Y) :-
    es_estudiante(X),
    es_estudiante(Y),
    X \= Y.
```

1.3 Aritmética en Prolog

Prolog usa el operador `is` para evaluar expresiones aritméticas:

```
cuadrado(X, R) :- R is X * X.

factorial(0, 1) :- !.
factorial(N, F) :-
    N > 0,
    N1 is N - 1,
    factorial(N1, F1),
    F is N * F1.
```

Consultas y resultados de ejemplo

```
?- factorial(5, F).
F = 120.

?- es_joven(X).
X = ana ;
X = maria ;
X = luis.

?- estudia_logica(X).
X = ana ;
X = maria.
```

1.4 Listas en Prolog

Las listas son fundamentales en Prolog. Se representan como `[Cabeza|Cola]`:

```
mi_member(X, [X|_]).
mi_member(X, [_|T]) :- mi_member(X, T).

mi_longitud([], 0).
mi_longitud(_|T, N) :-
    mi_longitud(T, N1),
    N is N1 + 1.
```

Segunda Sesión: Continuación de Programación con Prolog

2.1 Operaciones Avanzadas con Listas

La sesión 2 ([sesion2_avanzado.pl](#)) profundiza en algoritmos sobre listas y estructuras de datos.

Quicksort en Prolog

```
quicksort([], []).
quicksort([H|T], Sorted) :-
    partition(H, T, Less, Greater),
    quicksort(Less, SortedLess),
    quicksort(Greater, SortedGreater),
    append(SortedLess, [H|SortedGreater], Sorted).

partition(_, [], [], []).
partition(Pivot, [H|T], [H|Less], Greater) :-
    H =< Pivot, !,
    partition(Pivot, T, Less, Greater).
partition(Pivot, [H|T], Less, [H|Greater]) :-
    H > Pivot,
    partition(Pivot, T, Less, Greater).
```

Resultado:

```
?- quicksort([3,1,4,1,5,9,2,6], Sorted).
Sorted = [1, 1, 2, 3, 4, 5, 6, 9].
```

Aplanar listas anidadas

```
aplanar([], []).
aplanar([H|T], R) :-
    is_list(H), !,
    aplanar(H, FH),
    aplanar(T, FT),
    append(FH, FT, R).
aplanar([H|T], [H|R]) :-
    aplanar(T, R).
```

Resultado:

```
?- aplanar([1,[2,[3,4]],5], F).
F = [1, 2, 3, 4, 5].
```

2.2 Árboles Binarios de Búsqueda

Los árboles se representan como términos: `nil` (vacío) o `t(Raiz, Izq, Der)`.

```
insertar_arbol(X, nil, t(X, nil, nil)).
insertar_arbol(X, t(R, I, D), t(R, NI, D)) :-
    X < R, !,
    insertar_arbol(X, I, NI).

inorden(nil, []).
inorden(t(R, I, D), Lista) :-
    inorden(I, LI),
    inorden(D, LD),
    append(LI, [R|LD], Lista).
```

Resultado:

```
?- insertar_arbol(5, nil, A1),
    insertar_arbol(3, A1, A2),
    insertar_arbol(7, A2, A3),
    inorden(A3, L).
L = [3, 5, 7].
```

2.3 Fibonacci con Memorización

```
:- dynamic fib_memo/2.

fibonacci(0, 0) :- !.
fibonacci(1, 1) :- !.
fibonacci(N, F) :-
    N > 1,
    (   fib_memo(N, F) -> true
    ;   N1 is N-1, N2 is N-2,
        fibonacci(N1, F1), fibonacci(N2, F2),
        F is F1 + F2,
        assertz(fib_memo(N, F))
    ).
```

2.4 Gramáticas DCG

Las **Definite Clause Grammars** (DCG) permiten definir gramáticas de forma natural:

```
oracion --> sujeto, predicado.
sujeto  --> determinante, sustantivo.
predicado --> verbo, objeto.
```

```
determinante --> [el] ; [la] ; [un] ; [una].
sustantivo --> [gato] ; [perro] ; [niño].
verbo --> [come] ; [corre] ; [ve].
```

Resultado:

```
?- phrase(oracion, [el, niño, ve, la, niña]).
true.
```

Tercera Sesión: Aplicaciones con Prolog

3.1 Las Torres de Hanoi

Descripción del problema

Las Torres de Hanoi consisten en tres postes y un conjunto de discos de diferentes tamaños. El objetivo es mover todos los discos del poste origen al poste destino siguiendo estas reglas:

1. Solo se puede mover un disco a la vez.
2. Solo se puede tomar el disco superior de cada poste.
3. No se puede colocar un disco más grande sobre uno más pequeño.

Solución en Prolog ([torres_hanoi.pl](#))

La elegancia de Prolog radica en que la solución recursiva es directa:

```
hanoi(1, Origen, Destino, _) :-
    format("Mover disco 1 de ~w a ~w~n", [Origen, Destino]).

hanoi(N, Origen, Destino, Auxiliar) :-
    N > 1,
    N1 is N - 1,
    hanoi(N1, Origen, Auxiliar, Destino), % mover N-1 al auxiliar
    format("Mover disco ~w de ~w a ~w~n", [N, Origen, Destino]),
    hanoi(N1, Auxiliar, Destino, Origen). % mover N-1 al destino
```

Traza de ejecución para N=3

```
=====
TORRES DE HANOI - 3 Disco(s)
=====
Secuencia de movimientos:
Paso 1: Mover disco 1 de [izquierda] --> [derecha]
Paso 2: Mover disco 2 de [izquierda] --> [centro]
Paso 3: Mover disco 1 de [derecha] --> [centro]
```

```
Paso 4: Mover disco 3 de [izquierda] --> [derecha]
Paso 5: Mover disco 1 de [centro] --> [izquierda]
Paso 6: Mover disco 2 de [centro] --> [derecha]
Paso 7: Mover disco 1 de [izquierda] --> [derecha]
```

```
Total de movimientos realizados: 7
Movimientos mínimos ( $2^3 - 1$ ): 7
```

Análisis de complejidad

La fórmula para el número mínimo de movimientos es $2^n - 1$, donde n es el número de discos:

Discos	Movimientos mínimos
1	1
2	3
3	7
4	15
5	31
10	1,023
20	1,048,575

3.2 El Mono y la Banana

Descripción del problema

Este es un problema clásico de **planificación en Inteligencia Artificial**:

- Un mono está en la **puerta** de una habitación.
- Hay bananas colgando del techo en el **centro** de la habitación.
- Una caja está en la **ventana**.
- El mono puede: caminar, empujar la caja, subirse a la caja, agarrar las bananas (solo si está en el centro sobre la caja).

Representación del estado

```
estado(PosMonico, EnCaja, PosCaja, TieneBanana)
```

- **PosMonico:** puerta | ventana | centro
- **EnCaja:** si | no
- **PosCaja:** puerta | ventana | centro
- **TieneBanana:** si | no

Estado inicial: `estado(puerta, no, ventana, no)`

Estado objetivo: `estado(_, _, _, si)`

Operadores del problema (`mono_banana.pl`)

```
% El mono puede caminar si NO está sobre la caja
puede_hacer(
    estado(Desde, no, PosCaja, Banana),
    caminar(Desde, Hasta),
    estado(Hasta, no, PosCaja, Banana)).

% El mono puede empujar la caja si está en el mismo lugar
puede_hacer(
    estado(Lugar, no, Lugar, Banana),
    empujar_caja(Lugar, Destino),
    estado(Destino, no, Destino, Banana)).

% El mono puede agarrar la banana si está en el centro sobre la caja
puede_hacer(
    estado(centro, si, centro, no),
    agarrar_banana,
    estado(centro, si, centro, si)).
```

Solución encontrada por Prolog

```
=====
  EL MONO Y LA BANANA - Solución
=====
Estado inicial: estado(puerta, no, ventana, no)

¡Solución encontrada!

Plan de acciones:
  Paso 1: caminar(puerta, ventana)
  Paso 2: empujar_caja(ventana, centro)
  Paso 3: subir_caja
  Paso 4: agarrar_banana

Total de pasos: 4
¡El mono obtuvo las bananas!
```

Análisis del problema

El motor de Prolog aplica **búsqueda por backtracking** para explorar el espacio de estados. Gracias al operador `\+` (negación por fallo), se evita visitar estados ya explorados. Esta es una demostración de cómo el paradigma lógico puede resolver problemas de planificación de forma declarativa.

Conclusiones

Ventajas del Paradigma Lógico

1. **Declaratividad:** Se describe *qué* se quiere, no *cómo* obtenerlo. El ejemplo del Mono y la Banana es prueba clara: solo se definen los estados y operadores, y Prolog encuentra el plan automáticamente.
2. **Backtracking automático:** La búsqueda de soluciones es intrínseca al lenguaje, sin necesidad de implementar algoritmos de búsqueda explícitamente.
3. **Unificación poderosa:** Permite coincidencia de patrones complejos de forma natural.
4. **Ideal para IA y NLP:** Problemas de planificación, sistemas expertos, procesamiento de lenguaje natural (DCG) y representación del conocimiento.

Desventajas

1. **Curva de aprendizaje pronunciada:** Requiere cambiar radicalmente la forma de pensar comparado con la programación imperativa.
2. **Eficiencia:** El backtracking puede ser costoso en problemas grandes si no se usan cortes (!) o memorización.
3. **Depuración compleja:** Trazar la ejecución lógica puede ser difícil.

Comparación con otros paradigmas

Característica	Imperativo	Funcional	Lógico
Unidad base	Instrucción	Función	Cláusula lógica
Flujo de control	Explícito	Composición	Backtracking
Estado	Mutable	Inmutable	Sin estado
Ejemplo	C, Java	Haskell, Erlang	Prolog

Archivos del Proyecto

```
prolog_practica/
├── sesion1/
│   └── sesion1_introduccion.pl    # Hechos, reglas, aritmética, listas
├── sesion2/
│   └── sesion2_avanzado.pl        # Listas, árboles, Fibonacci, DCG
├── sesion3/
│   ├── torres_hanoi.pl           # Problema de las Torres de Hanoi
│   └── mono_banana.pl            # Problema del Mono y la Banana
├── reporte.md                    # Este reporte (fuente Markdown)
└── reporte.pdf                   # Reporte compilado en PDF
```

Instrucciones de Ejecución

```
# Instalar SWI-Prolog (Ubuntu/Debian)
sudo apt install swi-prolog

# Sesión 1
swipl sesion1/sesion1_introduccion.pl
?- factorial(5, F).
?- estudia_logica(X).

# Sesión 2
swipl sesion2/sesion2_avanzado.pl
?- quicksort([3,1,4,1,5,9,2,6], S).
?- fibonacci(10, F).

# Sesión 3 - Torres de Hanoi
swipl sesion3/torres_hanoi.pl
?- resolver_hanoi(3).

# Sesión 3 - Mono y Banana
swipl sesion3/mono_banana.pl
?- resolver.
?- resolver_detallado.
```

Referencias

- Bratko, I. (2001). *Prolog Programming for Artificial Intelligence* (3ra ed.). Pearson Education.
- Clocksin, W. F. & Mellish, C. S. (2003). *Programming in Prolog* (5ta ed.). Springer.
- SWI-Prolog Documentation. (2024). Recuperado de <https://www.swi-prolog.org/pldoc/>
- Russell, S. & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4ta ed.). Pearson.